

ME 6000 Numerical Analysis - Assignment for Numerical Errors

Taylor series of $\exp(x)$: $e^x = \sum_{k=0}^N \frac{x^k}{k!}$

For $x = -10$ and $x = +10$, $N = 10, 30, 50$, this script performs:

- (1) double precision, increasing k ($k = 0, 1, 2, \dots, N$)
- (2) double precision, decreasing k ($k = N, N-1, \dots, 0$)
- (3) single precision, increasing k
- (4) single precision, decreasing k

and compares every result with the exact value $\exp(x)$.

NOTE on how the terms are formed

Computing $x^k/\text{factorial}(k)$ literally overflows in single precision:

for $x = -10$, $N = 50$ we would need 10^{50} and $50! \sim 3.04e64$, while the largest single-precision number is only $\sim 3.40e38 \rightarrow \text{Inf}/\text{Inf} = \text{NaN}$.

We therefore build the terms with the stable recurrence

$$t_0 = 1, \quad t_k = t_{k-1} * x / k$$

which is mathematically identical but keeps every intermediate value inside the representable range ($\max |t_k| = 2755.7$ for $x = -10$).

The rounding behaviour we want to study is unaffected.

```
clear; clc; format long e

xList = [-10, 10];
NList = [10, 30, 50];

for x = xList
    exact = exp(x);                                     % reference ("exact") value
    fprintf('\n===== \n');
    fprintf(' x = %d      exact e^x = %.15e \n', x, exact);
    fprintf('===== \n');
    fprintf('%5s %10s %12s %24s %14s %14s \n', ...
            'N', 'precision', 'order', 'summation', 'abs. error', 'rel. error [%]');

    for N = NList
        % ---- the four required cases -----
        Sdi = taylorSum(x, N, 'double', 'increasing'); % (1)
        Sdd = taylorSum(x, N, 'double', 'decreasing'); % (2)
        Ssi = taylorSum(x, N, 'single', 'increasing'); % (3)
```

```
Ssd = taylorSum(x, N, 'single', 'decreasing'); % (4)
```

```
printRow(N, 'double', 'increasing', Sdi, exact);
printRow(N, 'double', 'decreasing', Sdd, exact);
printRow(N, 'single', 'increasing', Ssi, exact);
printRow(N, 'single', 'decreasing', Ssd, exact);
fprintf('%s\n', repmat('-', 1, 90));
```

```
end
```

```
end
```

```
=====
```

```
x = -10      exact e^x = 4.539992976248485e-05
```

```
=====
```

N	precision	order	summation	abs. error	rel. error [%]
10	double	increasing	1342.5873015873	1.3426e+03	2.95725e+09
10	double	decreasing	1342.5873015873	1.3426e+03	2.95725e+09
10	single	increasing	1342.58740234375	1.3426e+03	2.95725e+09
10	single	decreasing	1342.58752441406	1.3426e+03	2.95725e+09

30	double	increasing	0.000970341579791461	9.2494e-04	2037.32
30	double	decreasing	0.000970341579858314	9.2494e-04	2037.32
30	single	increasing	0.000862379791215062	8.1698e-04	1799.52
30	single	decreasing	0.0008697509765625	8.2435e-04	1815.75

50	double	increasing	4.53999296235717e-05	1.3891e-13	3.05977e-07
50	double	decreasing	4.53999295757512e-05	1.8673e-13	4.11308e-07
50	single	increasing	-6.25618267804384e-05	1.0796e-04	237.802
50	single	decreasing	1.52587890625e-05	3.0141e-05	66.3903

```
=====
```

```
x = +10      exact e^x = 2.202646579480672e+04
```

```
=====
```

N	precision	order	summation	abs. error	rel. error [%]
10	double	increasing	12842.3051146384	9.1842e+03	41.696
10	double	decreasing	12842.3051146384	9.1842e+03	41.696
10	single	increasing	12842.306640625	9.1842e+03	41.696
10	single	decreasing	12842.3056640625	9.1842e+03	41.696

30	double	increasing	22026.4640362589	1.7585e-03	7.98379e-06
30	double	decreasing	22026.4640362589	1.7585e-03	7.98379e-06
30	single	increasing	22026.466796875	1.0021e-03	4.54938e-06
30	single	decreasing	22026.46484375	9.5106e-04	4.31779e-06

50	double	increasing	22026.4657948067	7.2760e-12	3.30328e-14
50	double	decreasing	22026.4657948067	3.6380e-12	1.65164e-14

50	single	increasing	22026.46875	2.9552e-03	1.34166e-05
50	single	decreasing	22026.466796875	1.0021e-03	4.54938e-06

```

function S = taylorSum(x, N, prec, order)
% TAYLORSUM Partial Taylor sum of exp(x) up to k = N.
% prec : 'double' or 'single' -> precision of EVERY operation
% order : 'increasing' or 'decreasing' order of accumulation

cast = str2func(prec);          % @double or @single
xc   = cast(x);

% --- build the terms t_k = x^k/k! with the stable recurrence -----
t    = zeros(1, N+1, prec);
t(1) = cast(1);                % k = 0
for k = 1:N
    t(k+1) = t(k) * xc / cast(k); % t_k = t_{k-1}*x/k
end

% --- accumulate in the requested order -----
S = cast(0);
if strcmp(order, 'increasing')
    idx = 1:N+1;                % k = 0,1,...,N (smallest first)
else
    idx = N+1:-1:1;             % k = N,...,1,0 (smallest last)
end
for i = idx
    S = S + t(i);               % stays in the chosen precision
end
end

```

```

function printRow(N, prec, order, S, exact)

```

```
absErr = abs(double(S) - exact);  
relErr = absErr/abs(exact)*100;  
fprintf('%5d %-10s %-12s %24.15g %14.4e %14.6g\n', ...  
        N, prec, order, double(S), absErr, relErr);  
end
```